

MPS224 ASSIGNMENT: THE DAMPED PENDULUM EXPERIMENT

In this assignment you will analyse data from a damped pendulum experiment, use curve fitting to recover the physical parameters of the system, and then verify your results using signal processing techniques. You will need to make a Jupyter notebook with some code cells and some markdown cells following the instructions below.

You should submit your work in two files via Blackboard by the deadline shown there: as a `.ipynb` file, and also as a PDF. To produce the PDF, export your notebook to HTML via **File** → **Save and Export Notebook As** → **HTML**, then open the resulting `.html` file in a browser and print it to PDF (e.g. using **File** → **Print** → **Save as PDF**). Before submitting, restart the kernel and run all cells from top to bottom, and check that there are no errors and that all plots and output are visible.

1. PRELIMINARIES

1.1. Background. A pendulum hanging at angle θ (in radians) from the vertical, subject to friction, satisfies the equation

$$\ddot{\theta} + 2\gamma\dot{\theta} + \omega^2 \sin(\theta) = 0, \quad (\text{A})$$

where $\gamma \geq 0$ is the damping coefficient and $\omega > 0$ is the natural frequency, related to the length of the pendulum and the acceleration due to gravity by $\omega = \sqrt{g/\ell}$. Equation (A) has no closed-form solution for $\theta(t)$ in general.

However, when θ is small we have $\sin(\theta) \approx \theta$, and equation (A) is well-approximated by the *linearised equation*

$$\ddot{\theta} + 2\gamma\dot{\theta} + \omega^2 \theta = 0. \quad (\text{B})$$

This is a second-order linear ODE with constant coefficients, which can be solved exactly. In the *underdamped* case $\gamma < \omega$, the solution with initial conditions $\theta(0) = \theta_0$ and $\dot{\theta}(0) = \eta_0$ is

$$\theta(t) = \theta_0 c(t) + \left(\frac{\eta_0 + \gamma \theta_0}{\omega_d} \right) s(t), \quad (\text{C})$$

where $c(t) = e^{-\gamma t} \cos(\omega_d t)$ and $s(t) = e^{-\gamma t} \sin(\omega_d t)$, and $\omega_d = \sqrt{\omega^2 - \gamma^2}$ is the *damped natural frequency*.

1.2. Data files. Two data files are provided on Blackboard: `pendulum1.csv` and `pendulum2.csv`. They contain measurements of $\theta(t)$ recorded from two different pendulum experiments. Both data sets are ‘noisy’ and the second data set also has some imperfections that you will need to address before fitting. Download the files and place them in the same directory as your notebook so that you can open these with `pd.read_csv()`. You will fit a linear model to the first data set, and a non-linear model to the second data set.

1.3. Imports. Near the top of your notebook you should have the following imports.

```
import numpy as np
import sympy as sp
import scipy as sc
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
```

You can add others as well if you find it necessary.

Task 1.1. Introduction. Write an introduction for your notebook in markdown cells. The introduction should explain the background to the assignment: what system is being modelled, what differential equation is being solved, what data will be analysed, and what methods will be used. It should be possible for someone to read your notebook and follow what you are doing without referring to this briefing. (You may wish to complete this task *after* some of the others).

2. VERIFYING THE EXACT SOLUTION

Task 2.1. Equation (C) is given to you above. Use `sympy` to verify that it is correct, i.e. that it satisfies equation (B) and the initial conditions. You should do this symbolically: declare t , γ , ω_d , θ_0 , η_0 as `sympy` symbols, define the expression on the right-hand side of (C), and check that:

- (1) substituting $t = 0$ gives θ_0 ,
- (2) the derivative at $t = 0$ gives η_0 , and
- (3) the expression satisfies equation (B).

For (iii), note that since you are working with ω_d as a symbol rather than ω , you can write $\omega^2 = \omega_d^2 + \gamma^2$, so the equation to check is $\ddot{\theta} + 2\gamma\dot{\theta} + (\omega_d^2 + \gamma^2)\theta = 0$. You can use `sp.simplify()`, `sp.trigsimp()` or `sp.expand()` to check whether an expression is zero; for trigonometric expressions `sp.trigsimp()` is often the most reliable.

Write a markdown cell above this task explaining what you are doing and why.

Task 2.2. Write three functions:

```
cos_exp(ts, gamma, omega):
sin_exp(ts, gamma, omega):
plot_exact(gamma, omega, theta0, eta0, t_max=20, num_points=500):
```

The first two functions should evaluate the functions $c(t)$ and $s(t)$, respectively. The third function evaluates equation (C) at `num_points` equally spaced times between 0 and `t_max`, and plots θ against t . The plot should have size 10×4 , with labelled axes. The third function should call the first two.

Call `plot_exact` with $\gamma = 0.5$, $\omega = 3$, $\theta_0 = 1$ and $\eta_0 = 0$ to verify that the result looks like an oscillation that decays over time. Add a markdown cell below to serve as a caption; briefly explain what the plot shows, and state the parameter values.

3. FITTING A LINEAR MODEL

The first data file `pendulum1.csv` is a set of experimental measurements of the angle $\theta(t)$ as a function of time t , for a pendulum with a **known** natural frequency $\omega = 1$ and damping coefficient $\gamma = 1/10$. The data starts some time after the pendulum was released. Your task is to determine the initial parameters, θ_0 and η_0 , from this data.

Task 3.1. Read the first data file, `pendulum1.csv`, into a `pandas` DataFrame using `pd.read_csv()`. Inspect the first few rows with `.head()` and print the column names and data types. Make a labelled plot of the data, and add a markdown cell as a caption.

Task 3.2. Find best-fit values for θ_0 and η_0 by fitting a two-parameter linear model

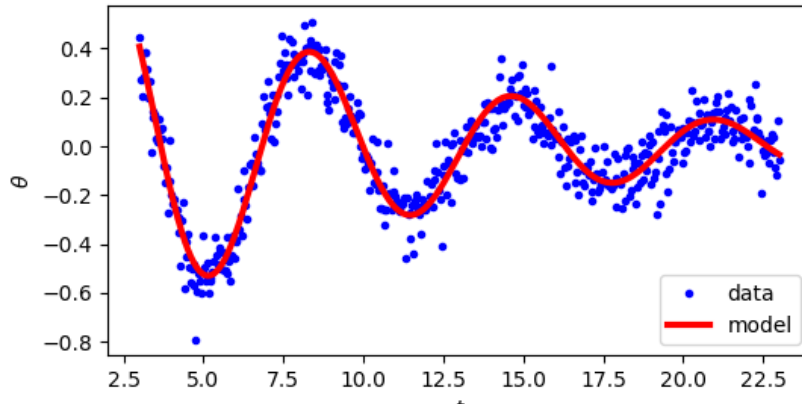
$$F(t; \beta_0, \beta_1) \equiv \beta_0 \phi_0(t) + \beta_1 \phi_1(t)$$

to the data set, by following the instructions below. Here β_0 and β_1 are parameters (which are related to θ_0 and η_0 in a way that you should determine), and $\phi_0(t) = c(t)$ and $\phi_1(t) = s(t)$ are the functions from parts 1 and 2. For more background on the theory of fitting linear models, see the slides from Lecture 8.

Follow these steps:

- (1) Make a 2D array $\mathbf{X}$ of dimension $N \times 2$, where N is the number of samples in the data file, and fill this with the values $X_{ij} = \phi_j(t_i)$. You should use the functions from part 2 for this (`cos_exp` and `sin_exp`).
- (2) Make a 2D array $\mathbf{A} = \mathbf{X}^T \mathbf{X}$ of dimension 2×2 , and a 1D array $\mathbf{b} = \mathbf{X}^T \boldsymbol{\theta}$. Here $\mathbf{X}^T$ denotes the transpose of $\mathbf{X}$, and $\boldsymbol{\theta}$ is a 1D array with the values of θ from the data file. (*Hint*: For matrix multiplication use `@`, instead of `*`).
- (3) Solve $\mathbf{A}\mathbf{x} = \mathbf{b}$ for $\mathbf{x}$ to find the two parameters β_0 and β_1 .
- (4) Hence deduce best-fit values of θ_0 and η_0 .

Make a plot showing the best-fit model curve overlaid onto the data. Add labels and a caption. It may look something like this:



You may wish to extend the model curve on the left up to $t = 0$, so that we can see the initial value θ_0 .

Write a markdown cell describing how the fitting was carried out, and state the best-fit values of θ_0 and η_0 to 3 decimal places.

4. PARAMETER ESTIMATION BY CURVE FITTING

In the second pendulum experiment, none of the four parameters (γ , ω_d , θ_0 and η_0) are known *a priori*, and our task is to find these by fitting a non-linear model.

Task 4.1. Read the second data file, `pendulum2.csv`, into a `DataFrame`, and then **clean** the data as follows.

- (a) The file may contain rows with missing values. Remove these using `.dropna()`.
- (b) Some rows may contain values of $|\theta|$ that are implausibly large (greater than $\pi/2$), which are likely measurement errors. Remove these rows.
- (c) Convert the cleaned data to `numpy` arrays for use in later tasks.

After cleaning, make a plot of θ against t . Write a markdown cell describing the shape of the data and whether it looks consistent with an underdamped oscillation.

Task 4.2. We will use `sc.optimize.curve_fit()` to fit equation (C) to the cleaned data and thereby estimate the parameters γ , ω_d , θ_0 and η_0 .

Write a function `exact_model(t, gamma, omega_d, theta0, eta0)` that returns the right-hand side of equation (C), evaluated at the array of times `t`, for the given parameter values. (Note that this function takes ω_d directly as a parameter, rather than ω ; this makes the fitting simpler.)

Pass `exact_model` to `sc.optimize.curve_fit()` together with your cleaned time and θ arrays. It is important to supply reasonable initial guesses for the parameters via the `p0` argument. You can estimate these from your plot: θ_0 is approximately the value of θ at the start of the cleaned data; η_0 is close to zero if the pendulum starts nearly at rest; ω_d can be estimated from the approximate period of oscillation T in the

plot via $\omega_d \approx 2\pi/T$; and γ is small and positive. The function returns the estimated parameters and their covariance matrix; print the estimated values of γ , ω_d , θ_0 and η_0 .

Plot the cleaned data together with the fitted curve on the same axes, with appropriate labels and a legend, so that you can assess visually how well the fit works.

Write a markdown cell above this task explaining the method, and a markdown cell below commenting on the quality of the fit. The latter cell should include the best-fit values for γ , ω , θ_0 and η_0 stated to three decimal places.

Task 4.3. From your estimate of ω_d and γ , compute $\omega = \sqrt{\omega_d^2 + \gamma^2}$ and hence state the estimated pendulum length $\ell = g/\omega^2$, where $g = 9.81 \text{ ms}^{-2}$. Comment on whether this value seems physically reasonable.

5. SIGNAL PROCESSING

Task 5.1. As an independent check, use the discrete Fourier transform to estimate the dominant oscillation frequency directly from the second dataset. (See lecture 9 for more about the theory).

Apply `np.fft.rfft()` to your cleaned θ array. Use `np.fft.rfftfreq()` to obtain the corresponding frequencies (you will need to pass the time-step Δt as the `d` argument). Plot the magnitude of the Fourier coefficients against frequency, with labelled axes.

Identify the frequency f^* at which the magnitude is largest. Convert this to an angular frequency $\omega_d^* = 2\pi f^*$ and compare with the value of ω_d obtained by curve fitting in Task 4.2. Write a markdown cell commenting on the agreement between the two estimates. Note that because the signal is decaying rather than perfectly periodic, the peak in the spectrum may be somewhat broad; this is expected. (In fact, there is a link between the width of the peak and the value of γ ; investigate this if you wish).

6. FURTHER EXPLORATION

This section is open-ended. You should write code and provide explanation in markdown cells, using L^AT_EX where appropriate. 20% of the marks for this assignment are reserved for this section. Some suggestions for tasks are below. Attempt no more than two tasks. You may define your own tasks if you wish (if you define your own tasks, please take care to describe them fully in markdown cells).

- **Residual analysis** [Easier task]. Compute the residuals between the fitted curve and the cleaned data. Plot them against time. Do they look like random noise, or is there systematic structure? What might cause any structure you observe?
- **Fitting the nonlinear equation** [Moderate task]. For larger oscillations, equation (C) is only an approximation. Write a function that solves the full nonlinear equation (A) numerically using `sc.integrate.solve_ivp()`, with a given set of parameters, and returns θ at the same time points as the data. Use `sc.optimize.minimize()` or `sc.optimize.curve_fit()` to fit this to the data. How do the estimated parameters compare with those from Task 4.2?
- **Stroboscopic plots (driven pendulum)** [Harder task]. Consider the driven case $f(t) = \alpha \cos(\Omega t)$, which corresponds to shaking the pendulum at an external frequency Ω . The full equation becomes

$$\ddot{\theta} + 2\gamma\dot{\theta} + \omega^2 \sin(\theta) = \alpha \cos(\Omega t). \quad (\text{D})$$

Rewrite (D) as a first-order system in (θ, η) where $\eta = \dot{\theta}$, and use `sc.integrate.solve_ivp()` to solve it numerically. When solving, discard an initial transient by solving from t_0 to t_2 but only recording output from t_1 onwards; this ensures the solution has settled into its long-term behaviour. Suggested defaults: $\gamma = 1/4$, $\omega = 1$, $\Omega = 2/3$, $\alpha = 1$, $t_0 = 0$, $t_1 = 500$, $t_2 = 1000$.

Once you have a solver working, build a *stroboscopic plot*: instead of recording the solution at uniformly spaced times, record it only at multiples of the forcing period $T = 2\pi/\Omega$. Plot each sampled point (θ_i, η_i) as a dot, folding θ into the range $(-\pi, \pi]$ first. The horizontal axis should have markings at $0, \pm\pi/2$ and $\pm\pi$.

Experiment with different values of α (try values between 1.6 and 2.2) and produce at least one plot that you find interesting. Describe what you observe.

7. GUIDANCE

Submission. Submit two files on Blackboard by the deadline shown there:

- (1) Your Jupyter notebook as a `.ipynb` file.
- (2) A PDF version produced by exporting to HTML and printing to PDF (see the first part of this document for instructions).

Before submitting, restart the kernel and run all cells in order. Check that there are no errors and that all plots are visible.

Notebook structure.

- Please organise your notebook into sections with headings that match the sections of this brief. Use exactly the function names and signatures specified here.
- Your notebook should be self-contained: a reader should be able to follow it without referring to this briefing.
- All functions should have a docstring describing inputs and outputs.
- For code that implements a non-obvious algorithm, add comments to explain how it works.
- Functions should not print or plot anything unless this brief specifically says they should. If a function is supposed to produce output, call it so that the output is visible in the submitted notebook.
- Do not redefine any function after it has first been defined. If you need a modified version, give it a new name.
- Your code should not create or save any files. It should read `pendulum_data.csv` as instructed, but should not write any output files.
- Remove any `input()` statements before submission.

Marking. 15% of the marks are for presentation and organisation. This includes:

- Quality of commentary and explanation in markdown cells.
- Organisation into sections with headings and subheadings.
- Well-structured L^AT_EX for mathematics.
- Consistent formatting conventions (code in backticks, italics or bold for defined terms, etc.).
- Spelling and grammar.

A further 20% is reserved for Section 6 (Further exploration).

Academic integrity.

- You may discuss the assignment with other students in general terms, but you must not copy any code or text from them. Write your own notebook in your own words. Mention any collaboration in your Acknowledgements.
- You may use Generative AI assistants to help you write or understand code. However, you are responsible for ensuring that all code and text in your notebook is correct: AI-generated code can contain subtle errors, so test every function carefully.
- You must record your use of Generative AI in your Acknowledgements: state which tool(s) you used and describe how.
- You may search the internet for information; list all sources in your Acknowledgements with specific URLs.
- Please include two final sections in your notebook. **Acknowledgements:** record all sources, collaborators, and AI usage here. **Sandbox:** use this for scratch work; it will not be marked.
- Please include your registration number in your notebook.
- Do not include your name in your notebook.